997. Find the Town Judge

Easy | Topics | Companies

In a town, there are `n` people labeled from `1` to `n`. There is a rumor that one of these people is secretly the town judge.

If the town judge exists, then:

1. The town judge trusts nobody.
2. Everybody (except for the town judge) trusts the town judge.
3. There is exactly one person that satisfies properties **1** and **2**.

You are given an array `trust` where `trust[i] = [a_i, b_i]` representing that the person labeled `a_i` trusts the person labeled `b_i`. If a trust relationship does not exist in `trust` array, then such a trust relationship does not exist.

Return *the label of the town judge if the town judge exists and can be identified, or return* `-1` *otherwise.*

Example 1:

> **Input:** n = 2, trust = [[1,2]]
> **Output:** 2

Example 2:

> **Input:** n = 3, trust = [[1,3],[2,3]]
> **Output:** 3

Example 3:

> **Input:** n = 3, trust = [[1,3],[2,3],[3,1]]
> **Output:** -1

Constraints:

- `1 <= n <= 1000`
- `0 <= trust.length <= 10^4`
- `trust[i].length == 2`
- All the pairs of `trust` are **unique**.
- `a_i != b_i`
- `1 <= a_i, b_i <= n`

DS / Day-89 / 5-11-26

Leetcode-997 = find the town judge, / /

Imagine there is a town with n peoples. They are numbered: 1, 2, 3, ---, n.
the town has a secret judge.
we don't know who the judge is. so we need to identify that person using the given trust array.

But who qualifies as a judge?
there are exactly 3 conditions:-
- the judge trusts nobody
- everybody else trusts the judge
- there must be exactly one person satisfying these conditions

Let's understand these conditions with a real-life example, suppose there are 3 people:-

People in the town:

1 person1 2 person2 3 person3.

suppose: trust = [[1,3], [2,3]]
this means:- [1,3] → person1 trusts person 3
[2,3] → " 2 " "

trust relationships:-

1 → 3
2 → 3

__/__/__

person 3 is the judge.

person 3 trusts nobody
person 1 " person 3
person 2 " person 3.

So, answer = 3.

* We don't need to know whom the judge trusts or what the judge does, we only need to check the two trust conditions.

* the function receives:-

```
int n;
vector<vector<int>>& trust;
```

n = total no. of people
trust = list of trust relationships.

for example, n = 3;

trust = {{1,3}, {2,3}};

every inner array contains only 2 values. [a,b]

means, person a trusts person b.

one imp thing: a trust relationship goes in one direction. if person 1 trusts person 3, it doesn't mean person 3 trusts person 1.

what do we need to calculate?

⇒ we need two qualities for every person: __/__/____

1. indegree, how many people trust this person?

for ex, Ej :-

trust = {{1,3}, {2,3}};

person 3 has an indegree of 2 but 2 people trust person 3.

2. outdegree, how many people does this person trust?

in the same example, person 3 has an outdegree of 0 but they trust nobody,

the judge must have:-

indegree = n-1
outdegree = 0] → final conditions.

why n-1? bcz if there are 3 people, the judge must be trusted by the others people.

if there are 5 people, the judge must be trusted by the other 4 people. The judge cannot trust anyone.

The Bruteforce can take O(n×m) t.c as, it will

n = number of people.

m = " " trust relationships.

check every person over by one & ask: 1. does everyone else trust this person. 2. does this person trust nobody?

not optimal, as suppose there are 1000 people & 10,000 trust relationships, we may examine those 10,000 relations 1000 times, that's unnecessary work.

So, Optimal ?

Instead of checking every person separately, we will maintain two arrays,

```
vector<int> indegree(n+1, 0);
vector<int> outdegree(n+1, 0);
```

Code :

```
class Solution{ public:
    int findJudge(int n, vector<vector<
    int>>& trust){
```

We receive, n = no. of people.
trust = trust relationships.

We return an int. representing the judge's label or -1.

creates an array of size n+1, initially, everybody has an indegree of 0,

```
vector<int> indegree(n+1, 0);
vector<int> outdegree(n+1, 0);
```

ex, indegree = {0,0,0,0};

creates outdegree arrays,

stores how many people each person trusts.

initially, outdegree = {0,0,0,0};

for (auto t : trust) { → traverses the trust array. We visit each trust relationship exactly once.

int a = t[0];
int b = t[1];

extract the two people,

remember, [a, b] means

a trusts b.

so: a = person who trusts
b = " being trusted.

Suppose,
trust = {{1,3}, {2,3}};
1 will be list {1, 3}, & then {2, 3}.

outdegree[a]++; → person a trusts somebody, therefore their outdegree increases by 1

indegree[b]++; } person b being trusted.

therefore, their indegree rise by 1.

→ Now that we have calculated everyone's indegree & outdegree, we check each person.

for (int i = 1; i <= n; i++) {

finding the judge.

if (indegree[i] == n-1 && outdegree[i] == 0) {

everyone else trusts this person.

this person trusts nobody.

return i;

} return ans;

} return -1; } we found the judge, so return their label.

} no judge exists, return -1, nobody satisfied both conditions.

T.C = O(n + m)
↳ no. of people.
↳ no. of trust relationships.

S.C = O(n)

→ 1st loop visits all m trust relationships,
time = O(m).
→ 2nd loop visits all n people,
time = O(n),
total = O(m) + O(n)
= O(m+n).

Why is this optimal? In the brute-force approach, we repeatedly scan the same trust relationships for each people.

but is the optimal approach:-
→ we read each trust relationship exactly once
→ we immediately update the two relevant people's count.
→ after processing all relationships, we already know everyone's trust status.
→ we simply check which person satisfies the judge conditions.

We avoid repeating work here,
so T.C = O(n×m) → reduces to O(m+n)

anyone (optimal): n=3, trust = {{1,3}, {2,3}, {3,1}};

we have 3 relationships:-

1. person 1 trusts person 3
2. person 2 " " 3
3. " 3 " " 1.

initially:

indegree = {0,0,0,0};

outdegree = {0,0,0,0};

1st iteration = {1,3};

outdegree[1]++;

indegree[3]++;

why size 4? bcz

we have n=3, 1,2,3

we leave index 0

unused.

{0,0,0,1} = indegree

{0,1,0,0} = outdegree

2nd iteration = {2,3};

outdegree[2]++;

indegree[3]++;

answer:-

indegree = {0,0,0,2};

outdegree = {0,1,1,0};

2 red t = {3,1};

outdegree[3]++;

indegree[1]++;

final indegree = {0,1,0,2};

outdegree = {0,1,1,1};

now let's check everyone,

person	indegree	outdegree	result
1	1	1	no
2	0	1	no
3	2	1	no

for person 3;

indegree[3] == 2 // true

outdegree[3] == 0 // false.

person 3 trusts person 1, so

" " can't be the judge.

no one satisfies both conditions.

o/p = -1

Can we solve it using only one array?
Yes, there's another elegant approach using a single score array.
Instead of storing indegree & outdegree separately, we combine them.

```
class solution { public:
  int findJudge(int n, vector<vector<int>>& trust) {
    vector<int> score(n+1, 0);
    for (auto t : trust) {
      score[t[0]]--;
      score[t[1]]++; }
    for (int i=1; i<=n; i++) {
      if (score[i]==n-1) {
        return i; } } return -1; } };
```

whenever a person trusts somebody, score[t[0]]--; , we subtract 1 from the person who trusts,
whenever somebody trusts a person: score[t[1]]++;
we add 1 to the person being trusted,
so, score[i] = indegree[i] - outdegree[i]
for a judge, score[i] = (n-1) - 0
therefore: score[i] = n-1
so we only need to check, if (score[i] == n-1).
why it is safe? ⇒ the max. possible indegree = n-1,
min. outdegree = 0.
so, a scores of n-1 is only possible when;
indegree = n-1, outdegree = 0,
this is also O(n+m) time & O(n) space with one array only.

Accepted 92 / 92 testcases passed

Akankshyaa submitted at Nov 05, 2026 12:52

Analysis Solution

Save 41%

Big Diwali Dhamaka

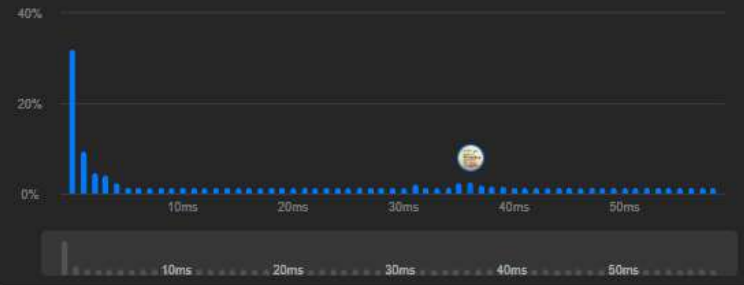
Unlock the Full Experience with Premium for ₹583.25/mo — Ends Nov 8!

Runtime

36 ms | Beats 21.08%

Memory

73.34 MB | Beats 13.68%



Code | C++

```cpp
class Solution {
public:
    int findJudge(int n, vector<vector<int>>& trust) {
        vector<int> indegree(n+1,0);
        vector<int> outdegree(n+1,0);
        for(auto t:trust){
            int a= t[0];
```

```cpp
class Solution {
public:
    int findJudge(int n, vector<vector<int>>& trust) {
        vector<int> indegree(n+1,0);
        vector<int> outdegree(n+1,0);
        for(auto t:trust){
            int a= t[0];
            int b= t[1];
            outdegree[a]++;
            indegree[b]++;
        }
        for(int i=1;i<=n; i++){
            if(indegree[i]==n-1 && outdegree[i]==0){
                return i;
            }
        }
        return -1;
    }
};
```

Saved

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

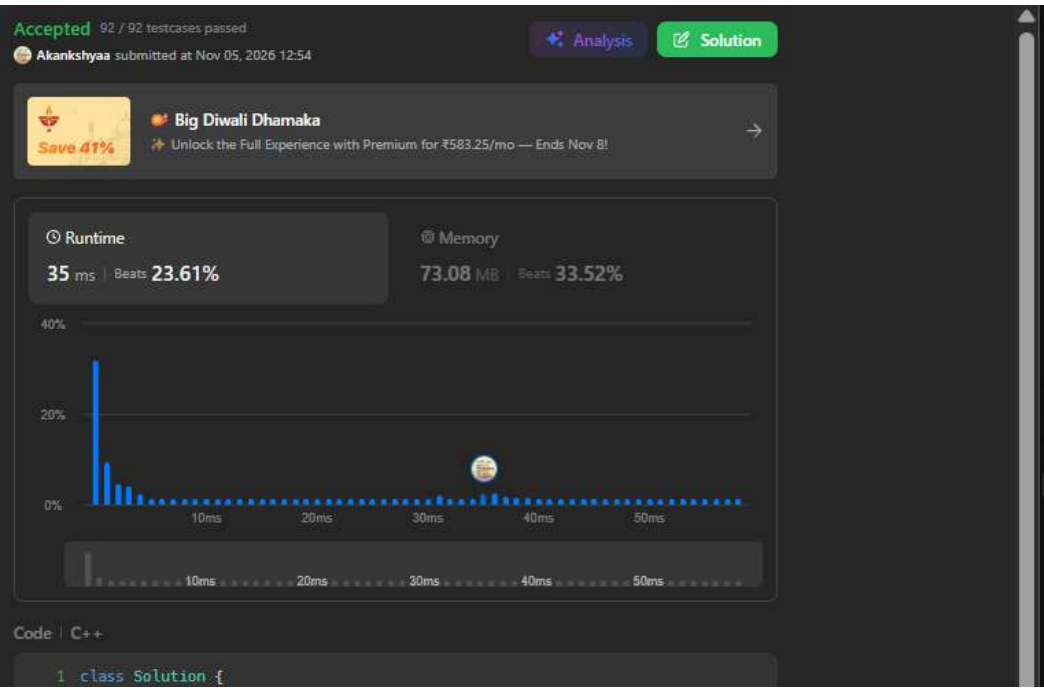
Input

n =

2

trust =

[[1,2]]


Accepted 92 / 92 testcases passed
Akankshyaa submitted at Nov 05, 2026 12:54
Analysis
Solution
Save 41%
Big Diwali Dhamaka
Unlock the Full Experience with Premium for ₹583.25/mo — Ends Nov 8!
Runtime
35 ms | Beats 23.61%
Memory
73.08 MB | Beats 33.52%
40%
20%
0%
10ms
20ms
30ms
40ms
50ms
Code | C++
1 class Solution {


1 class Solution {
2 public:
3 int findJudge(int n, vector<vector<int>>& trust) {
4 vector<int> score(n + 1, 0);
5 for(auto t : trust) {
6 score[t[0]]--;
7 score[t[1]]++;
8 }
9 for(int i = 1; i <= n; i++) {
10 if(score[i] == n - 1) {
11 return i;
12 }
13 }
14 return -1;
15 }
16 };
Saved
Testcase | Test Result
Accepted Runtime: 0 ms
Case 1
Case 2
Case 3
Input
n =
2